

INTERNATIONAL BLACK SEA UNIVERSITY
School of Computer Science · Computer Science Program

A.L.F.R.E.D.

Adaptive Local-First Routing and Execution Distillation
for Small Language Models

David Zoidze

Bachelor's Thesis in Computer Science

Supervisor: **Giorgi Merabishvili**

Tbilisi · July 2026

The problem: easy tasks, huge models

You say: **"set an alarm for 7:00."**

Today, an agent system answers that with a billion-dollar habit: send it to the biggest model available and let it think.

WHAT THE BIG-MODEL HABIT COSTS

It doesn't fit — a model big enough to reason reliably does not run on a phone

It's slow and costs money — seconds and a cloud bill, for a task a human does with one tap

It leaks — the request carries personal context that never needed to leave the device

And it is still wrong — we will show that even a 35-billion-parameter model fails on real phone settings

Setting an alarm is an easy task. **Why does it need a model that can write poetry?**

This thesis asks: how little model does a phone action actually need?

The observation: these tasks are repetitive

Look at what people actually ask a phone to do. The same requests, over and over, with one word changing:

THE REQUEST

"set an alarm for 7:00"

"set an alarm for 8:30"

"set an alarm for noon"

THE COMMAND

clock alarm -t 7:00

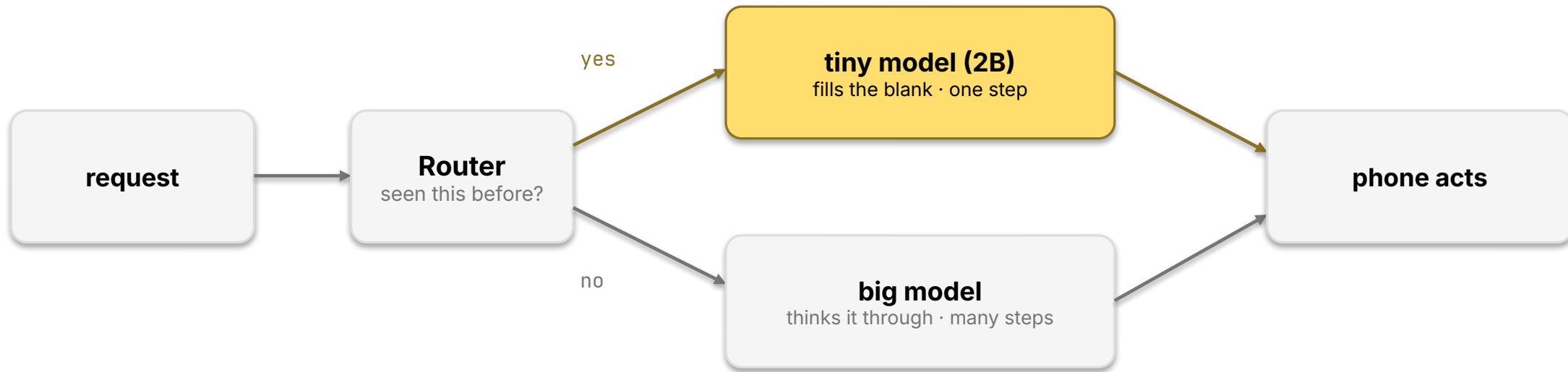
clock alarm -t 8:30

clock alarm -t noon

The command never changes shape. Only the blank changes. And there is exactly one right answer — the alarm is set for 7:00 or it is not.

So here is the idea: **make the command shape a fixed, deterministic template — and let a tiny model do the only creative part: filling in the blank.**

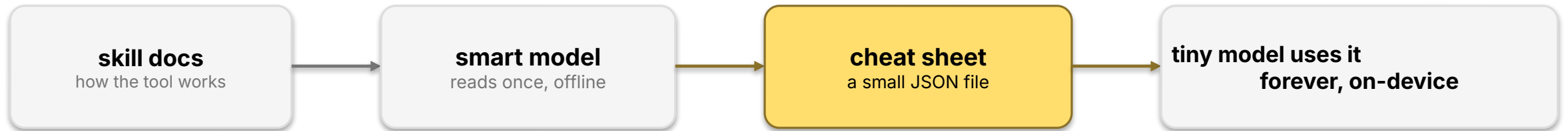
The idea: a reflex for the routine, a brain for the rest



- **It is the reflex-vs-thinking split from psychology** — practiced actions run fast and automatically; novel problems get slow, deliberate thought. We give the phone the same two speeds.
- **The tiny model is never asked to be clever** — it gets the template and the request, and extracts one thing: the value for the blank. It cannot invent a wrong command — the shape is fixed.
- **The big model is reserved for the rare, genuinely new request** — multi-step routines, vague asks, anything without a template. That is where thinking earns its cost.

Where do the templates come from? Write them once.

Nobody hand-writes templates at scale. We have a smart model write them — once, offline, like a cheat sheet.



- ▶ **The cheat sheet holds the knowledge, so the model doesn't have to** — the command shape, what goes in each blank, and sanity checks on the result
- ▶ **It is a file you can read** — the usual way to compress a big model is to bake it into a smaller model's weights — a black box. A JSON file can be inspected, corrected, and shared. Adding a capability = adding a file.
- ▶ **Pay once, use thousands of times** — the smart model's cost is paid at writing time, not every time you set an alarm
- ▶ **We play fair** — the smart model never sees our test tasks or answers, and a written cheat sheet is frozen — no quiet fixing after seeing what failed

How we tested it (without fooling ourselves)

A FAKE PHONE

A sandboxed settings backend

every task runs on a fresh copy of the phone's state; the copy is destroyed afterward — no real device, no network, nothing persists

JUDGE THE OUTCOME, NOT THE WORDS

Did the phone end up right?

a command can look perfect and do the wrong thing — so we only check the final state, and fail the task if anything else was touched

THREE DIFFICULTY LEVELS

From friendly to realistic

409 one-shot tasks across 7 tools · a clean, helpful settings interface · a faithful copy of Android's messy one, which accepts wrong values silently

EVERY NUMBER IS A SAVED RUN

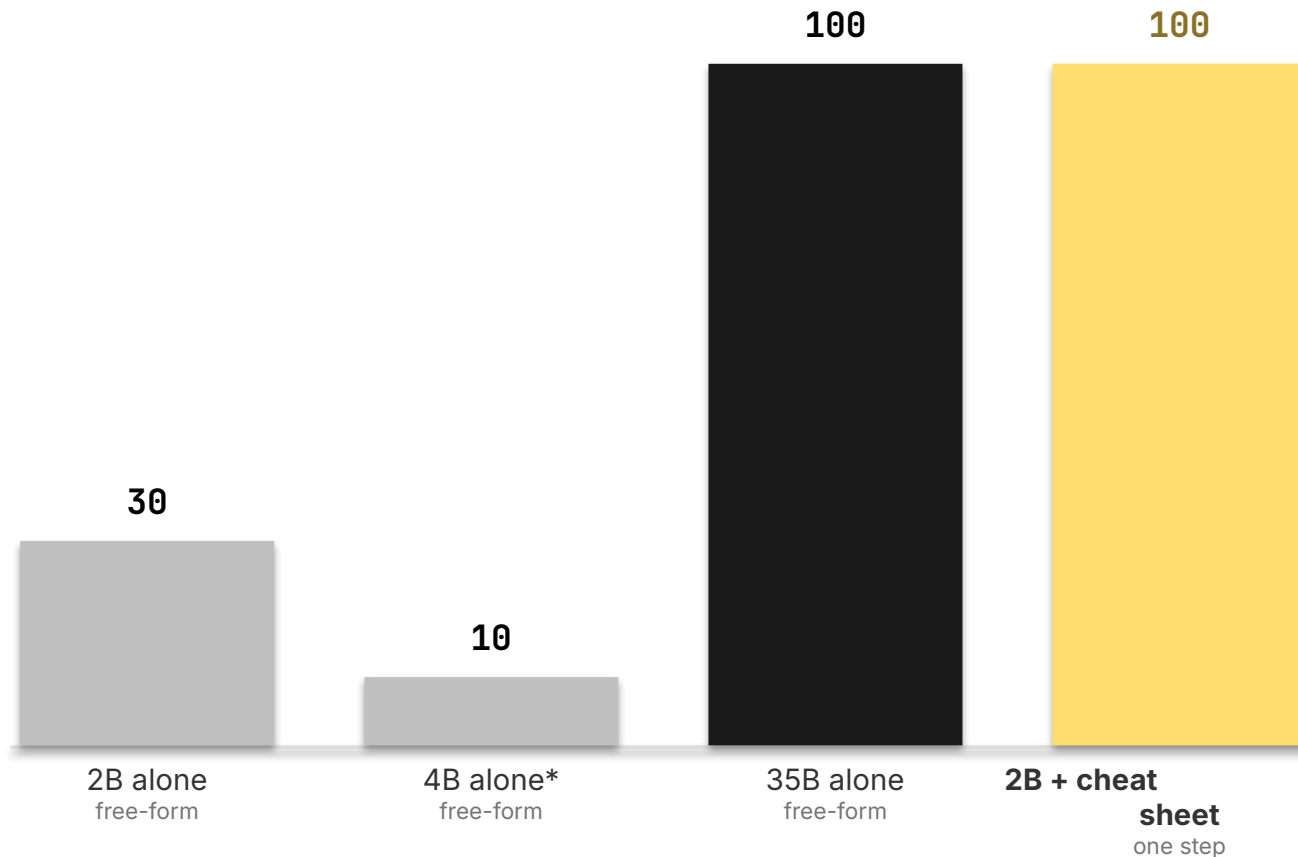
No hand-edited results

each figure cites the log file it came from; models run one at a time; all models are quantized to fit a real phone's memory

To measure the executor alone, routing is assumed correct in these tests — the router is measured separately (slide 9).

Result 1 — The cheat sheet replaces 17× the parameters

Same tasks, same scoring. How often does each setup produce the correct command?



*4B ran with a small context window (a known confound) — the point: size alone is not a reliable lever

100% = 100%

the tiny model with a cheat sheet matches the 35B model that reasons freely — at ~17× fewer parameters

<1 s vs 6.4 s

one step on the phone, versus a multi-step conversation that has to run in the cloud

72/72 vs 14/72

holds on the full task set too: with the sheet, everything passes; without it, the same tiny model gets 1 in 5

Result 2 — Why even the big model fails on a real phone

On realistic Android settings, the 35B model scores only **62.5%**. Its mistakes reveal that “hard” is really two different problems:

FINDING THE RIGHT KNOB

Which setting controls this?

This is searchable. The big model explores the interface, lists the settings, and finds the right one. Being smarter helps.

✓ **solved by scale**

KNOWING THE SECRET CODE

What value does it expect?

“Total silence” is the number 2. “Largest font” is the string 1.30. That is written nowhere. The model guesses — and guesses wrong the same way every time.

✗ **no amount of thinking helps**

Real example: “DND, total silence” → correct: **zen_mode 2** the 35B answered: 1 — every single time it was asked.

The secret codes live in the cheat sheet. With it, the tiny model reaches 80% — beating the 35B that reasons from scratch.

What we can claim — and what is still broken

THE TEACHER MATTERS

80% vs 60%

- cheat sheets written by a strong cloud model: the tiny model passes 80% of tasks
- written by a mid-size local model: 60% — it bakes its own wrong guesses into the sheet
- the tiny model faithfully executes whatever it is given, right or wrong — so write the sheet with the best teacher you have

THE MATH WORKS OUT

pays off after ~10 uses

- writing a cheat sheet costs roughly 10,000 tokens of a smart model, once
- each later use saves ~1,000 tokens and 5+ seconds, and runs on the phone
- for anything you do daily, that is a 100–1000× win; one-off requests just go to the big model

STILL BROKEN: THE PICKER

66–85%

- executing a chosen template is basically solved (96–100%)
- choosing which template a request matches — the router — is not; it confuses similar-looking options
- we say this plainly: the router is now the open problem, and it is where our future work points

Future work — in order of how much each would help

1

Let the teacher look at the real phone

today the cheat-sheet writer works from documentation alone, so some secret codes stay guesses; give it read-only access to a real device and guesses become observed facts

2

Take arithmetic away from the model

converting “80% brightness” to the number 204 is math, not language — do it with a plain function inside the cheat sheet, so neither model can get it wrong

3

Fix the picker (the router)

the right template is almost always in the top 3 candidates; the failure is choosing between look-alikes — better selection prompting, or a small trained chooser

4

Let cheat sheets improve from their own mistakes

run the sheet, watch what fails on a practice device, rewrite, repeat 2–3 times — a mediocre teacher climbs toward cloud quality; the on-phone part stays a frozen single step

5

Learn new cheat sheets from the user

if you do the same three-step routine every night, that repetition can be mined into a new template — plus privacy-safe sharing of sheets between users

All of it is planned inside one small, transparent agent framework (Pi) — the executor and the big-model loop share a single runtime.

Thank you.

Questions?

David Zoidze

A.L.F.R.E.D.: Adaptive Local-First Routing and Execution Distillation for Small Language Models

International Black Sea University · Tbilisi · July 2026